Technical Report: Star Type Classification, ML and Deep Learning Pipeline

Jashlein Leanne T. Marquez

April 2026

Dataset source: Star Type Classification — Kaggle
https://www.kaggle.com/datasets/brsdincer/star-type-classification

Abstract

This report presents a comprehensive machine learning and deep learning pipeline for the automated classification of star types using six stellar physical features. The analysis is applied to the Star Type Classification dataset from Kaggle, which contains 240 samples across six stellar classes: Brown Dwarf, Red Dwarf, White Dwarf, Main Sequence, Supergiant, and Hypergiant. Four classification models were implemented and evaluated — Random Forest, Gradient Boosting, Support Vector Machine (RBF kernel), and a Tensorflow/Keras Neural Network — within an interactive Tkinter GUI application that supports live training monitoring, model comparison, and manual prediction. Results demonstrate near-perfect or perfect classification accuracy (1.0000) on test data for the best-performing models, with Absolute Magnitude and Radius identified as the most discriminative features. The report documents the full pipeline from data preprocessing and exploratory analysis to model architecture, training procedures, evaluation metrics, and limitations.

Keywords: Star Classification, Machine Learning, Deep Learning, Random Forest, Neural Network, Hertzsprung-Russell Diagram, Tkinter GUI, Scikit-learn, TensorFlow

1 Introduction

The classification of stars into distinct physical types is a foundational task in astrophysics, forming the basis for understanding stellar evolution, galactic structure, and the life cycles of stars. Since the early twentieth century, astronomers have used observable stellar properties — such as temperature, luminosity, spectral class, and color — to categorize stars into a systematic taxonomy. The Hertzsprung-Russell (HR) diagram, which plots stellar luminosity against surface temperature, remains one of the most powerful tools in this effort, revealing distinct groupings that correspond to evolutionary stages.

With the advent of large-scale astronomical surveys and modern computing, machine learning methods have increasingly been applied to automate and accelerate stellar classification. Unlike traditional rule-based approaches, supervised learning algorithms can identify complex, non-linear boundaries between stellar classes directly from data, enabling rapid and consistent predictions across massive catalogs.

This project implements an end-to-end classification pipeline for the Star Type Classification dataset published on Kaggle by Dincer (2021). The dataset contains 240 stellar observations described by six features: effective surface temperature (K), luminosity relative to the Sun ($L_\odot$), radius relative to the Sun ($R_\odot$), absolute visual magnitude, dominant color, and Harvard spectral class. Stars are labeled across six categories spanning a wide range of the HR diagram: Brown Dwarfs, Red Dwarfs, White Dwarfs, Main Sequence stars, Supergiants, and Hypergiants.

The pipeline is delivered as an interactive desktop application built with Tkinter, embedding Matplotlib charts directly into the GUI. Users can explore the dataset through multiple visualization panels, configure and train four distinct classifiers — Random Forest, Gradient Boosting, SVM (RBF), and a Keras Neural Network — observe live training curves, compare model performance on a shared test set, and classify new stellar observations by entering physical parameters manually.

This report documents the full technical workflow: dataset structure and preprocessing, exploratory data analysis, model architectures and hyperparameter choices, evaluation results, feature importance analysis, and a discussion of findings and limitations.

2 Objectives

The project was guided by the following primary objectives:

- Perform exploratory data analysis on stellar physical parameters to characterize the six star classes and identify key discriminative features.
- Preprocess the dataset through categorical encoding, log-transformation of skewed physical features, and standardization for use with gradient-based and distance-based classifiers.
- Implement and train four classification models — Random Forest, Gradient Boosting, SVM (RBF kernel), and a deep Keras Neural Network — using a consistent 80/20 train-test split with stratified sampling.
- Evaluate all models on a held-out test set using accuracy, per-class F1 score, 5-fold cross-validation, and confusion matrix analysis.
- Identify the most important features driving classification performance through Random Forest feature importance scores.
- Deliver a fully interactive GUI application allowing users to configure training parameters, monitor live training progress, compare models, and classify novel stellar observations.

3 Methodology

3.1 Dataset Source and Structure

The Star Type Classification dataset was obtained from Kaggle and is stored as a CSV file named Stars.csv. The dataset contains 240 rows and 7 columns, with each row representing one stellar observation and the final column providing the ground-truth class label.

Column	Description	Type	Example Values
Temperature	Surface	Integer	3068, 9700, 30000

	temperature (K)		
L	Luminosity (L⊙)	Float	0.0024, 1.0, 500000
R	Radius (R⊙)	Float	0.17, 1.0, 1000
A_M	Absolute Magnitude	Float	16.12, 4.8, −10.6
Color	Dominant stellar color	String (cat.)	Red, Blue, White, Yellow
Spectral_Class	Harvard spectral type	String (cat.)	M, K, G, F, A, B, O
Type	Star type label (0–5)	Integer	0=Brown Dwarf, …, 5=Hypergiant

Table 1. Dataset features, types, and example values

3.2 Class Distribution

The dataset is balanced, with 40 samples per class across all six star types. This deliberate balance eliminates class imbalance as a confounding factor and ensures that accuracy directly reflects discriminative capability rather than majority-class bias.

Type ID	Class Name	Count	Typical Temp. Range (K)
0	Brown Dwarf	40	< 3,000
1	Red Dwarf	40	2,500 – 5,000
2	White Dwarf	40	5,000 – 40,000+
3	Main Sequence	40	3,000 – 40,000
4	Supergiant	40	3,000 – 35,000
5	Hypergiant	40	10,000 – 40,000+

Table 2. Star type labels, names, sample counts, and typical temperature ranges.

3.3 Feature Chracteristics

The physical features span many orders of magnitude. Luminosity ranges from approximately 0.0001 $L_\odot$ (Brown Dwarfs) to over 500,000 $L_\odot$ (Hypergiants), and radius ranges from 0.01 $R_\odot$ to over 1,500 $R_\odot$. These extreme right-skewed distributions motivate the log-transformation applied during preprocessing. Absolute Magnitude ranges from roughly −10 (very bright supergiants and hypergiants) to +20 (dim brown dwarfs), encoding luminosity information on a logarithmic astronomical scale.

Spectral class follows the Harvard sequence O–B–A–F–G–K–M, ordered from hottest to coolest, and is strongly correlated with temperature. Dominant color is a categorical label that also correlates with temperature but exhibits some variability due to observational and nomenclature differences in the source data.

3.4 Data Preprocessing

All preprocessing steps are encapsulated in the load_and_preprocess() function in star_classification.py. The pipeline follows four sequential stages:

3.5 Categorical Encoding

The Color column undergoes text normalization before encoding: leading/trailing whitespace is stripped, the string is lowercased, hyphens are replaced with spaces, and double spaces are collapsed. Both Color and Spectral_Class are then transformed using scikit-learn's LabelEncoder, which assigns integer codes based on alphabetically sorted unique values. The encoded columns are named Color_enc and Spectral_Class_enc.

3.6 Log-Transformation

The three physical magnitude features — Temperature, Luminosity (L), and Radius (R) — are transformed with log1p (i.e., log(1 + x)) to compress their extreme dynamic ranges and produce distributions that are more amenable to standardization and gradient-based optimization. Absolute Magnitude is not log-transformed as it is already on a logarithmic astronomical scale and spans both positive and negative values.

3.7 Feature Matrix

The three physical magnitude features — Temperature, Luminosity (L), and Radius (R) — are transformed with log1p (i.e., log(1 + x)) to compress their extreme dynamic ranges

and produce distributions that are more amenable to standardization and gradient-based optimization. Absolute Magnitude is not log-transformed as it is already on a logarithmic astronomical scale and spans both positive and negative values.

Idx	Feature	Transform Applied	Source Column
0	Temperature (K)	log1p	Temperature
1	Luminosity ($L_\odot$)	log1p	L
2	Radius ($R_\odot$)	log1p	R
3	Absolute Magnitude	None	A_M
4	Color (encoded)	LabelEncoder	Color
5	Spectral Class (encoded)	LabelEncoder	Spectral_Class

Table 3. Feature matrix column index, transformations, and source columns.

3.8 Standardization and Splitting

After feature construction, all six columns are standardized to zero mean and unit variance using scikit-learn's StandardScaler, fit on the entire dataset prior to splitting. The

data is then partitioned into an 80% training set (192 samples) and a 20% test set (48 samples) using train_test_split with stratify=y to maintain class proportions in both splits. The random seed is fixed at 42 for reproducibility.

3.9 Random Forest

The Random Forest model is an ensemble of 200 decision trees trained with bootstrap sampling (n_estimators=200, random_state=42). Each tree is grown to full depth, and the final prediction is determined by majority vote across all trees. Random Forest is included as a strong baseline due to its robustness to feature scale (though features are pre-scaled for comparability), built-in feature importance estimation, and strong out-of-the-box performance on tabular datasets.

3.10 Gradient Boosting

The Gradient Boosting classifier uses an ensemble of 200 shallow decision trees built sequentially, where each tree corrects the residual errors of the previous ensemble (n_estimators=200, learning_rate=0.1, random_state=42). Gradient Boosting minimizes a differentiable loss function (cross-entropy for multiclass) through gradient descent in function space, producing a highly accurate ensemble at the cost of longer training time compared to Random Forest.

3.11 Support Vector Machine (RBF Kernel)

The SVM uses a Radial Basis Function (Gaussian) kernel with a regularization parameter C=10 and gamma='scale' (i.e., 1 / (n_features * X.var())). The RBF kernel maps the feature space into an infinite-dimensional Hilbert space, enabling the SVM to construct non-linear decision boundaries. The multi-class problem is handled using the standard one-vs-one strategy implemented in scikit-learn. Probability calibration is enabled (probability=True) to support per-class confidence estimates in the prediction interface.

3.12 Neural Network (TensorFlow/Keras)

The neural network is a feedforward multi-layer perceptron built with Keras. The default configuration uses three hidden layers with 128, 64, and 32 units respectively, ReLU activations, batch normalization after each hidden layer, and dropout (rate=0.3) for regularization. A softmax output layer with 6 units produces class probability distributions.

Layer	Units / Type	Details
Input	6	Standardized 6-feature vector
Dense 1	128 (ReLU)	L2 regularization (λ = 1e-4)
BatchNorm 1	—	Normalizes activations across batch
Dropout 1	0.3	Randomly drops 30% of units during training
Dense 2	64 (ReLU)	L2 regularization (λ = 1e-4)
BatchNorm 2	—	
Dropout 2	0.3	

Dense 3	32 (ReLU)	L2 regularization (λ = 1e-4)
BatchNorm 3	—	
Dropout 3	0.3	
Output	6 (Softmax)	Multi-class probability distribution

Table 4. Default neural network layer configuration.

The network is compiled with the Adam optimizer (learning rate = 0.001) and sparse categorical cross-entropy loss. Training uses a 15% validation split from the training set, with two Keras callbacks: EarlyStopping (monitors validation loss, patience=20, restores best weights) and ReduceLROnPlateau (halves the learning rate when validation loss plateaus for 10 consecutive epochs, minimum LR = 1e-5). The default number of epochs is 150 with a batch size of 16.

4 Results and Analysis

4.1 Test Set Accuracy

All four models achieve exceptional test accuracy on this dataset, with both Random Forest and Gradient Boosting achieving perfect 1.0000 accuracy on the 48-sample held-out test set. The SVM (RBF) and Neural Network also achieve high accuracy in practice. These results reflect the high discriminability of stellar features once proper preprocessing is applied.

Model	Test Accuracy	CV Mean (±2σ)	CV Std
Random Forest	1.0000	~0.99	Low
Gradient Boosting	1.0000	~0.99	Low
SVM (RBF)	~0.98+	~0.97	Low
Neural Network (TF)	~0.97+	N/A	N/A

Table 5. Model performance summary on the held-out test set.

4.2 Confusion Matrix Analysis

The confusion matrix for the Random Forest model (as shown in Image 5 of the GUI reference) displays a perfect 8×8 diagonal pattern across all six classes, confirming that no misclassifications occurred on the test set. Each class (Brown Dwarf, Red Dwarf, White Dwarf, Main Sequence, Supergiant, Hypergiant) has exactly 8 test samples, and all are correctly classified. This result demonstrates that the six stellar types are effectively linearly or non-linearly separable in the log-transformed, standardized feature space.

4.3 Per-Class F1 Scores

Per-class F1 scores for the Random Forest model are uniformly 1.00 across all six star types, consistent with the perfect confusion matrix. This outcome indicates that even minority or borderline stellar classes — such as White Dwarfs, which have atypical temperature-luminosity combinations relative to their spectral class — are fully distinguished by the ensemble model without any precision-recall trade-off.

4.4 Feature Importance

Feature importance analysis from the Random Forest model reveals a clear ranking of discriminative power:

Rank	Feature	Approx. Importance	Interpretation
1	Absolute Magnitude (A_M)	~0.33	Most discriminative
2	Radius (R)	~0.31	Strong size signal
3	Luminosity (L)	~0.16	Correlated with A_M
4	Spectral Class	~0.11	Moderate

5	Temperature	~0.07	Correlated with spectral class
6	Color	~0.02	Least discriminative

Table 6. Random Forest feature importance ranking

Absolute Magnitude and Radius dominate the importance scores, together accounting for over 60% of the model's splitting decisions. This aligns with astrophysical intuition: brightness and physical size are the most direct observables that distinguish stellar populations on the HR diagram. Color, by contrast, contributes minimally — likely because its information is already captured by Temperature and Spectral Class.

5 Discussion

The near-perfect classification results are attributable to several converging factors. First, the six stellar classes occupy largely non-overlapping regions of the feature space: Brown and Red Dwarfs have low luminosity and small radii; White Dwarfs occupy a unique low-luminosity, potentially high-temperature region; Main Sequence stars follow the well-known mass-luminosity-temperature relationship; while Supergiants and Hypergiants have enormous radii and extreme luminosities. Once log-transformed, these differences become linearly or quasi-linearly separable.

Second, the dataset is balanced (40 samples per class), small (240 total), and was likely curated to represent archetypical examples of each stellar type, reducing within-class variance. Real-world stellar catalogs would contain transition objects, measurement noise, and boundary cases that could substantially reduce classification accuracy.

The feature importance results reveal a degree of redundancy: Temperature, Spectral Class, and Color are all correlated measures of stellar surface conditions, while Luminosity, Absolute Magnitude, and Radius relate to stellar size and intrinsic brightness. The model concentrates most of its discriminative power in Absolute Magnitude and Radius, effectively using Luminosity and Temperature as secondary confirmatory features.

5.1 Limitations

Several important limitations should be acknowledged:

- **Dataset size:** 240 samples is extremely small for training deep learning models reliably. The Neural Network results should be interpreted with caution, as variance across random seeds may be substantial.
- **Curated data:** The dataset appears to contain archetypal representatives of each class rather than a random astronomical sample. Performance on real survey data (e.g., Gaia DR3, SDSS) would likely be lower.
- **Categorical encoding:** LabelEncoder assigns arbitrary integer codes to color names, implying a false ordinal relationship between colors (e.g., 'blue' < 'orange' < 'red' numerically). One-hot encoding or embeddings would be more principled.
- **Scaler leakage:** The StandardScaler is fit on the full dataset (X_scaled) before splitting, which constitutes minor data leakage. For larger datasets, this could introduce an optimistic bias in cross-validation scores.
- **Prediction encoding:** The manual prediction interface uses a hard-coded color-to-integer mapping that differs from the LabelEncoder fitted on training data, which may cause prediction discrepancies for uncommon color strings.

6 Conclusion

This project demonstrates that stellar classification from six physical features can be achieved with near-perfect accuracy using standard machine learning methods applied to a curated benchmark dataset. The Random Forest and Gradient Boosting classifiers both achieve 100% test accuracy, while the SVM and Neural Network deliver comparable performance. Feature importance analysis consistently identifies Absolute Magnitude and Radius as the most discriminative features, aligning with the physical intuition underlying the Hertzsprung-Russell diagram.

Future directions could include applying the pipeline to larger, noisier astronomical catalogs; exploring one-hot encoding for categorical features; implementing SHAP values for more interpretable feature attribution; and extending the classification to finer-grained spectral subtypes or variable star categories.

References

Dincer, B. (2021). Star Type Classification [Dataset]. Kaggle. https://www.kaggle.com/datasets/brsdincer/star-type-classification

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. https://www.tensorflow.org/

Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

Hertzsprung, E. (1911). On the Use of Photographic Effective Wavelengths for the Determination of Color Equivalents. Publikationen des Astrophysikalischen Observatoriums zu Potsdam, 22, 1–40.